

DAVA: Distributing Vaccines over Networks under Prior Information

September 25, 2024

Abstract

Given a graph, like a social/computer network or the blogosphere, in which an infection (or meme or virus) has been spreading for some time, how to select the k best nodes for immunization/quarantining immediately? Most previous works for controlling propagation (say via immunization) have concentrated on developing strategies for vaccination pre-emptively before the start of the epidemic. While very useful to provide insights into which baseline policies can best control an infection, they may not be ideal for making real-time decisions as the infection is progressing.

In this paper, we study how to immunize healthy nodes in the presence of already infected nodes. Efficient algorithms for such a problem can help public-health experts make more informed choices. First, we formulate the Data-Aware Vaccination problem, and prove it is NP-hard and also that it is hard to approximate. Secondly, we propose two effective polynomial-time heuristics DAVA and DAVA-fast. Finally, we also demonstrate the scalability and effectiveness of our algorithms through extensive experiments on multiple real networks, including epidemiology datasets, which show substantial gains of up to 10 times more healthy nodes at the end.

1 Introduction

Given a set of already infected people in a population, what are the healthy people who should be immediately given vaccines to best control the epidemic? How should Twitter decide which accounts to suspend/delete to stop rumors that are already present? Propagation-style processes on graphs/networks are powerful tools for modeling situations of interest in real life, like in social systems, cyber-security, and epidemiology. For example, diseases spreading over population contact networks, spam/rumors spreading on Facebook/Twitter, and malware spreading over computer networks are all propagation processes. As a result, manipulating and controlling such harmful propagation is an important and natural problem with numerous applications.

In this paper, we concentrate on how best to distribute vaccines to nodes on a network when the disease has already spread to certain parts of the graph. Intuitively, we want to study how to best build a 'wall' against an emerging and already spreading contagion. We assume the vaccines 'immunize' the nodes completely, i.e., they are removed from the network immediately. Most previous work in immunization tries to give algorithms and policies for pre-emptive immunization, where the goal is to find the best baseline policies for controlling an epidemic before it has started. However, in practice, this assumption is rarely true—specific individuals like travelers or those working with animals are more likely to be infected first, making the epidemic start from particular locations. Thus, while baseline policies provide useful guidelines, they may not be ideal for real-time decisions when an epidemic is already in progress. Efficient algorithms for such a Data-Aware Vaccination problem can help policymakers react and make better choices as conditions change. This problem has clear applications to social and cyber systems as well: which user accounts should be disabled in Facebook to stop an active spam message? Which computer nodes should install patches first in the presence of malware attacks?

Our main contributions include:

- **Problem Formulation and Hardness Results:** We formulate the Data-Aware Vaccination problem as a combinatorial optimization problem on arbitrary graphs, proving it is NP-hard and also hard to approximate.

- **Effective Heuristics:** We present effective polynomial-time heuristics DAVA and DAVA-fast for general graphs, as well as DAVA-tree, an optimal algorithm for m-trees.
- **Experimental Evaluation:** We provide extensive experiments on multiple real datasets, showing that our algorithms outperform others by up to 10 times in both magnitude and running time.

2 Preliminaries

This section presents the preliminaries. Table 1 lists the main symbols used in the paper. There is an underlying contact network G (between people/computers/blogs, etc.) on which the contagion (disease/virus/meme, etc.) spreads. For simplicity, we assume our graph is weighted and undirected, but all our methods and tools can be generalized to directed graphs.

Symbol	Definition and Description
DAV	Data-Aware Vaccination problem
SIR	Susceptible-Infected-Recovered Model
IC	Independent Cascade Model
$G(V, E)$	Graph with node set V and edge set E
I_0	Infected nodes set
$p_{u,v}$	Propagation probability from node u to v
δ	Curing probability for the SIR model
k	The budget (#nodes to give vaccines to)
S	Set of nodes selected for vaccination
$\sigma_G, I_0(S)$	Expected number of infected nodes at the end (footprint)
$\sigma'_G, I_0(S)$	Expected number of healthy nodes at the end
$\gamma_S(j)$	Expected benefit of $\sigma'_G, I_0(*)$ when adding j into S

Table 1: Terms and Symbols

We use two well-known virus propagation models: the Susceptible-Infected-Recovered (SIR) model and its special case, the Independent Cascade (IC) model. The SIR model has been widely used in epidemiology. Each node is in one of three states: susceptible (healthy), infected, or recovered. Infected nodes attempt to infect healthy neighbors with a probability $p_{u,v}$ at each time step, and infected nodes can recover with probability δ . Once recovered, a node no longer participates in the epidemic. The process starts with an initial infected set and ends when no infected nodes remain. The IC model differs in that each infected node gets precisely one chance to infect its neighbors.

3 Problem Formulation

We are given a set of nodes I_0 , which are already infected, and a graph $G(V, E)$ with node set V and edge set E . A healthy node can be vaccinated, removing it from the graph and preventing infection. The goal is to select the best set S of k healthy nodes to vaccinate to minimize the expected number of infected nodes during the entire process, $\sigma_G, I_0(S)$, given that I_0 nodes are already infected. Formally, the Data-Aware Vaccination problem is:

$$S^* = \arg \min_S \sigma_G, I_0(S), \quad \text{subject to } |S| = k \quad (1)$$

The alternative form of this problem is to maximize the expected number of healthy nodes at the end, $\sigma'_G, I_0(S)$, since minimizing the number of infected nodes is equivalent to maximizing the number of healthy ones.

4 Experiments

In this section, we give an experimental evaluation of our DAVA and DAVA-fast algorithms.

4.1 Experimental Setup

We describe our setup next.

4.1.1 Datasets

We run our experiments on multiple real datasets. In addition to trying to pick datasets of various sizes, we also chose them from different domains where the DAV problem is especially applicable. Table 2 shows the datasets we used.

Dataset	Model	Description
OREGON	IC	A Oregon AS router graph containing 633 links among 2172 AS peers.
STANFORD	IC	A Stanford CS hyperlink network containing 8929 nodes and 53829 links.
GNUTELLA	IC	A peer-to-peer network containing 39994 links among 10876 peers.
BRIGHTKITE	IC	A friendship network which consists of 58228 nodes and 214078 edges.
PORTLAND	SIR	A large urban social-contact graph used in national smallpox modeling studies. It contains
MIAMI	SIR	Another social-contact graph with about 0.6 million people and 2.1 million edges.

Table 2: Datasets used in our experiments

4.1.2 Settings

For the IC model, we use three illustrative settings:

- Uniform probability $p = 0.6$
- Uniform probability $p = 1$
- $p_{u,v}$ uniformly randomly chosen from $\{0.1, 0.5, 0.9\}$ (following literature [6])

We randomly choose 100 nodes to be infected initially as the set I_0 .

For the SIR model, since our socio-contact graphs have contact time between people [?], we use normalized contact time as the attack probability $p_{u,v}$, and set a uniform recovery rate $\delta = 0.6$. As the graphs are larger, we randomly choose 300 nodes to be infected initially as the set I_0 .

To get the expected number of healthy nodes after immunization, we run the processes 1000 times and take the average.

4.1.3 Baseline Algorithms

We compare our algorithms DAVA and DAVA-fast against various other competitors: RANDOM, DEGREE, PAGERANK, and NETSHIELD [?]. Note that NETSHIELD is a state-of-the-art pre-emptive immunization algorithm that aims to minimize the epidemic threshold of the graph. We take the top-k healthy nodes according to each algorithm.

All our experiments were conducted on a 4 Xeon E7-4850 CPU with 512GB of 1066Mhz main memory, and the algorithms were implemented in Python2.

4.2 Experimental Results

We describe the results of our experiments next. In short, our results demonstrate that DAVA and DAVA-fast get up to 10 times better solutions compared to the baselines (resulting in thousands of more healthy nodes). In addition, we also show the scalability of our algorithms on large datasets (DAVA-fast finishes within 20 minutes for our largest graphs). Finally, we show that DAVA-fast returns solutions that are very comparable to the ones returned by the more expensive DAVA.

4.2.1 Effectiveness for DAV

First of all, the number of the nodes in the first layer of the dominator trees of the graphs were a fraction (ranging from 0.3 to 0.7) of the total number of healthy nodes in the graph. As discussed before, because of Lemma 5.3, this reduces the solution space substantially.

Figure 1: Effectiveness for DAVA: IC model on various Real Datasets.

IC model Figure 3 shows our results for the IC model. In all networks, DAVA and DAVA-fast consistently outperform baseline algorithms.

As OREGON had only ~ 600 nodes, we varied k till 50 (roughly 9% of the graph). OREGON has a jellyfish-type structure, hence for lower k , most algorithms work well by targeting the nodes in the core. But for larger k , the periphery needs to be targeted, and here our algorithms provide the best solution. For bigger networks like GNUTELLA and STANFORD with tens of thousands of nodes, the difference in performance of DAVA and DAVA-fast from the other algorithms is clearer. PAGERANK and DEGREE perform well in STANFORD (a web-graph, with many hubs that these baselines exploit). Additionally, NETSHIELD performed well only in BRIGHTKITE, demonstrating that the type of solutions change drastically once we take the data of who is infected into account. In all these cases, DAVA and DAVA-fast performed the best.

Our faster heuristic, DAVA-fast, performed very well in all the networks, getting solutions almost as good as DAVA consistently. On STANFORD, DAVA-fast and DAVA both saved about 10 times more nodes than baseline algorithms, yet DAVA-fast took a fraction of the running time of DAVA (see Table 3). Also, the gains from our algorithms reduced as the p value decreased. This is expected: the weaker the disease (in spreading), the lower the benefit of vaccinating—i.e., the lower the savings gain from carefully selecting important nodes (as removing a node won’t further change the expected infections much).

Figure 2: Effectiveness for DAVA: SIR model on PORTLAND and MIAMI.

SIR model We got similar results under the SIR model on PORTLAND and MIAMI too, as shown in Figure 4. Since PORTLAND and MIAMI contain about 0.5 million nodes, DAVA was slow and could only allocate fewer than 100 nodes in 1 day, and hence we do not show it in the plots. (NETSHIELD took 1.5 days to allocate 2000 vaccines, while DAVA-fast finished in 20 minutes.) We notice that the larger k becomes, the better DAVA-fast performs than other algorithms: it saves more than 4000 nodes than the second-best algorithm PAGERANK, and more than 20,000 nodes than DEGREE (which is similar to the well-known acquaintance immunization method used in practice [?]).

4.2.2 Scalability

Although our algorithms are polynomial-time, we show some running time results to demonstrate scalability. Table 3 shows the running time for DAVA, DAVA-fast, and NETSHIELD when $k = 200$. DAVA-fast is much faster than DAVA—it took only 20 seconds to select 200 nodes in GNUTELLA, while DAVA takes more than an hour to finish it. For the large-scale datasets like PORTLAND and MIAMI, DAVA-fast took less than 15 minutes to select 200 nodes, while DAVA could not finish in the allotted time. Furthermore, DAVA-fast is up to 10 times faster than NETSHIELD—this is because NETSHIELD has a $O(nk^2)$ complexity (while our algorithms are linear in the budget).

Dataset	DAVA-fast	NETSHIELD	DAVA
OREGON	0.89	4.9	23.3
STANFORD	14.2	74.1	2920.4
GNUTELLA	20.1	79.4	4700.5
BRIGHTKITE	109.3	246.8	19444.1
PORTLAND	778.4	8211.6	-
MIAMI	1034.2	11233.9	-

Table 3: Running time (sec.) of NETSHIELD, DAVA, and DAVA-fast when $k = 200$. Runs terminated when running time $t > 10$ hours (shown by ‘-’).